FPGA INT8 MLP ACCELERATOR

Developer Setup Guide

From a fresh machine to passing testbenches, and how we work together

Team: Joey (`itsKannapi`), Teammate 2 (TBD), Teammate 3 (TBD) · Last updated 2026-10-22 · `https://github.com/itsKannapi/fpga-mlp-accelerator`

Short version

Clone the repo → install Vivado 2026.1 (Artix-7 only) and its free Basic license → in Vivado's Tcl console, `source <repo>/scripts/create_project.tcl` → Run Simulation → look for `ALL TESTS PASSED`. No Vivado (e.g. on a Mac)? Run `./scripts/run_sims.sh` with Icarus Verilog instead.

Contents

1 What you need **2**

2 Get the code **2**

3 Install Vivado 2026.1 **2**

4 Create the Vivado project **3**

5 Run the testbenches **3**

6 Python (only to retrain or re-export) **4**

7 Programming the board **4**

8 How we work together **4**

1 What you need

Item	Notes
Windows or Linux PC	**Vivado does not run on macOS.** Mac users can still run simulations with Icarus (Section 5.2) and do Python and docs work.
16 GB RAM, ~60 GB disk	For Vivado with a single device family installed.
Git	Windows: `winget install Git.Git`
Python 3.12+	Only needed to retrain or re-export weights. 3.14 works.
Basys 3 + micro-USB cable	Only for hardware steps. All simulation works without the board.

2 Get the code

```
git clone https://github.com/itsKannapi/fpga-mlp-accelerator.git
cd fpga-mlp-accelerator
git config --global user.name  "Your Name"
git config --global user.email "ID+username@users.noreply.github.com"
```

Use the `noreply` address from GitHub → Settings → Emails ("Keep my email addresses private"). Every commit's email is public in a public repo. Commits still count on your profile.

Clone somewhere with a short path with no spaces (e.g. `C:\dev\`), not a OneDrive-synced folder: OneDrive tries to sync Vivado's gigabytes of build files.

3 Install Vivado 2026.1

1. Download the AMD unified installer (free AMD account required).
2. Choose **Vivado**, Standard/Basic edition. In the device list, **deselect everything except 7 Series → Artix-7**. Skip Vitis. This keeps the install near 60 GB instead of 100 GB or more.
3. If you later search for part `xc7a35tcpg236-1` and get 0 matches, Artix-7 wasn't installed: *Help → Add Design Tools or Devices* (or re-run `xsetup.exe`) and add it.

3.1 License (required even for the free tier)

From 2026.1, Vivado will not start without a license file, including the free tier. The error is *"Vivado Design Suite cannot be launched because a valid license was not found."*

1. Open the Vivado License Manager → *View System Information*. Write down a Host ID (MAC address or disk serial).
2. On AMD's licensing site, select **"Vivado Basic Tier License, Node Locked License"** (Certificate, No Charge).
3. Lock it to the Host ID from step 1, **character for character**. A mismatched Host ID is the most common cause of the error above.

4. License Manager → *Load License* → select the `.lic` file. Check *View License Status* shows **HostID matches = YES**.

Do not pick these

"Start Now! 60 Day Trial" and **"Vivado Enterprise Tier, 60 Day Evaluation"** expire mid-project. **"HL WebPACK 2015 and Earlier"** only works for old tool versions. The Basic license renews yearly.

4 Create the Vivado project

The project folder `vivado/` is not in git. It is generated, and it would contain absolute paths from whoever made it. Build your own:

1. Start Vivado. At the start screen, open the **Tcl Console** tab at the bottom. (If a project is open, *File* → *Close Project* first.)
2. Run, with forward slashes:

```
source C:/dev/fpga-mlp-accelerator/scripts/create_project.tcl
```

3. It prints `Project ready:  ...mlp_accel.xpr`. The project is open.

The script writes `vivado/data_dir.vh` (where `rtl/` lives on *your* machine), adds every `src/*.v` and `sim/*.v` **without copying them**, sets the include paths, and sets the simulation to run to completion. Re-run it any time the project gets into a weird state, or after a teammate adds a new source file.

5 Run the testbenches

5.1 In Vivado

Flow Navigator → *Run Simulation* → *Run Behavioral Simulation*. Read the result in the Tcl Console (scroll up if needed). The script sets `tb_mlp_core` (the end-to-end test) as the default. To run a different testbench, type e.g. `set_property top tb_mac [get_filesets sim_1]` and run again.

Testbench	Expected last lines	Notes
`tb_mlp_core`	`ALL TESTS PASSED (2880 checks)`	also prints `ACCURACY : 99 / 100`
`tb_layer1`	`ALL TESTS PASSED (4259 checks)`	ReLU exercised 41×, clamp 2×
`tb_layer2`	`ALL TESTS PASSED (737 checks)`	
`tb_argmax`	`ALL TESTS PASSED (4024 checks)`	526 random tied cases
`tb_mac`	`ALL TESTS PASSED (2266 checks)`	

xsim takes from about 20 seconds (`tb_mac`) to a few minutes (`tb_mlp_core`). If you see `could not open golden.txt` or `cannot open include file`, the project wasn't made by the script: re-run

Section 4.

5.2 Without Vivado: Icarus Verilog

Fast, free, runs on macOS, Linux, and Git Bash on Windows. Use it as a quick check before every push.

```
brew install icarus-verilog        # macOS
sudo apt install iverilog          # Ubuntu
./scripts/run_sims.sh
```

It runs every `sim/tb_*.v`, prints one line per testbench, and exits non-zero if anything fails.

6 Python (only to retrain or re-export)

```
python -m venv .venv               # Windows: py -m venv .venv
.venv\Scripts\activate             # macOS/Linux: source .venv/bin/activate
pip install torch torchvision numpy
python quantize_export.py
```

Weights are frozen

Don't re-run `train_mlp.py`: retraining gives different weights and breaks every golden comparison. If `python` opens the Microsoft Store on Windows, use `py`, or turn off the `python.exe` alias under *Settings → Apps → Advanced app settings → App execution aliases*. Re-exporting currently needs a file only Joey has (open issue #1 in `progress.tex`).

7 Programming the board

1. Plug into the **PROG** USB port and turn on the board's power switch.
2. *Open Hardware Manager → Open Target → Auto Connect.*
3. Click **Program Device** and select the `.bit` file. Opening the target is not enough: the board shows "Programmed" even while it is still running the factory demo from flash.

Programming over JTAG is lost at power-off; reprogram each session. If Auto Connect finds nothing: check the cable carries data (not charge-only), try a port on the motherboard, or install Digilent Adept for its USB drivers.

8 How we work together

1. **Branch per task**: `git switch -c layer2-argmax`. Don't commit straight to `main`.
2. **Every new module comes with a self-checking testbench** that prints `ALL TESTS PASSED`, and a few planted bugs to prove it can fail (`design.tex` §4).
3. **Before opening a pull request**: `./scripts/run_sims.sh` passes, `progress.tex` has a

dated entry, and the PDFs are rebuilt.

4. **Pull request**: push the branch, open a PR on GitHub, and have one teammate review it before merging into `main`.
5. **Never commit** `vivado/`, `.venv/`, `data/`, or `*.npz`. `.gitignore` already blocks them; check `git status` before `git add`.
6. **Commit messages** say what changed and how it was verified, e.g. *"Add layer 2 + argmax; bit-exact vs golden on 64 images"*.

8.1 Building these documents

```
cd docs
latexmk -pdf design.tex progress.tex setup.tex
```

Needs a LaTeX distribution: MiKTeX on Windows, MacTeX on macOS, `texlive` on Linux. Or upload `docs/` to Overleaf. The PDFs are committed so they can be read on GitHub; rebuild and commit them whenever a `.tex` changes.